

Experiment No. 4

Title: Calculate the nth Fibonacci number efficiently

Aim: The aim of this program is to efficiently compute the nth Fibonacci number using techniques that optimize time complexity, ensuring that the computation is fast and scalable even for large values of n.

Theory: The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones.

starting from 0 and 1. In mathematical terms, the sequence is defined recursively as:

- $F(0) = 0$
- $F(1) = 1$
- $F(n) = F(n-1) + F(n-2)$ for $n > 1$

Experiment 4 Code:

```
def fibonacci_memo(n, memo):
    if n <= 1:
        return n

    if memo[n] != -1:
        return memo[n]

    memo[n] = fibonacci_memo(n - 1, memo) + fibonacci_memo(n - 2, memo)
    return memo[n]

def fibonacci_tab(n):
    if n <= 1:
        return n

    dp = [0] * (n + 1)

    dp[0] = 0
    dp[1] = 1

    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]

    return dp[n]

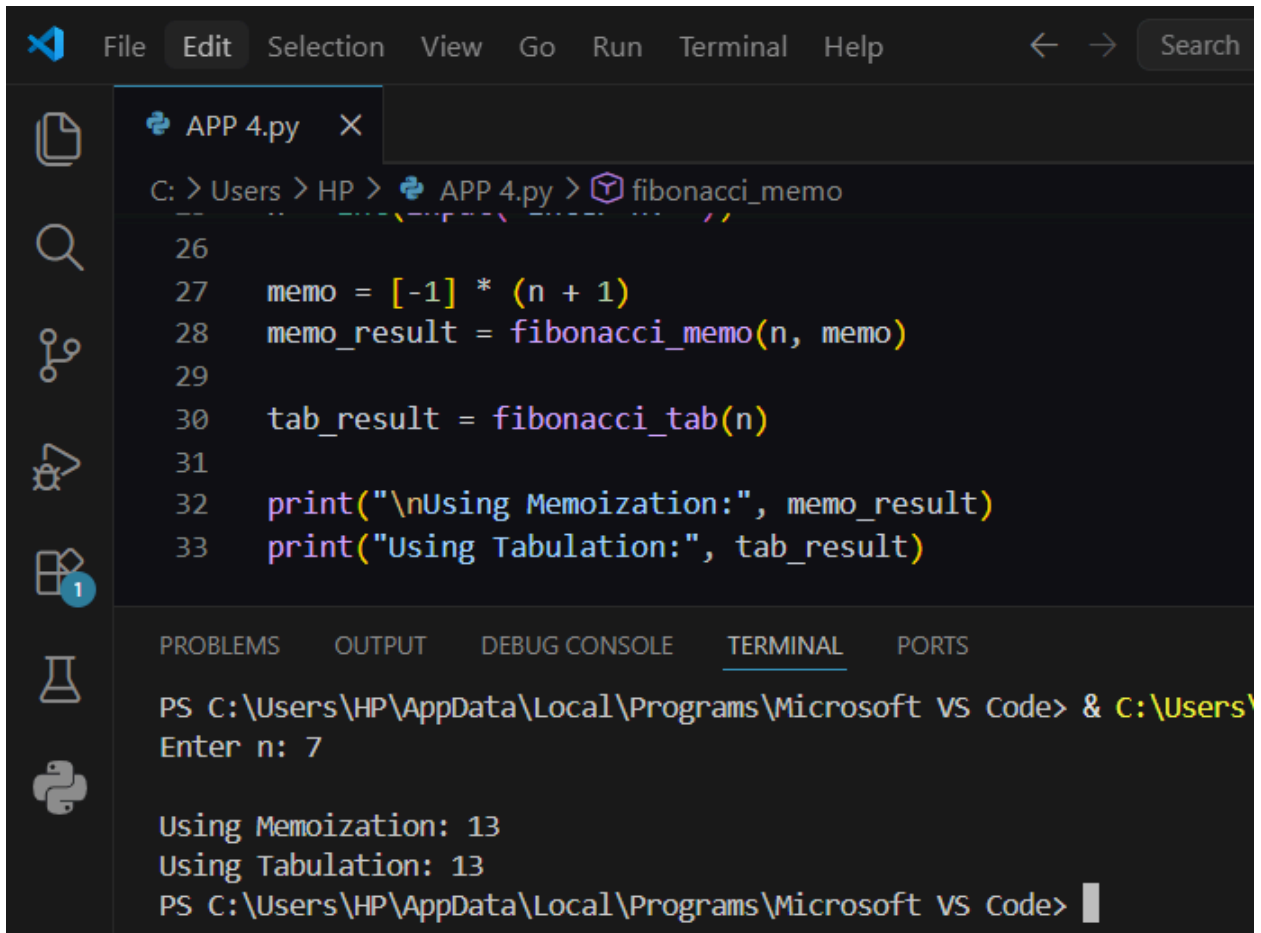
n = int(input("Enter n: "))

# Memoization
memo = [-1] * (n + 1)
memo_result = fibonacci_memo(n, memo)
```

```
# Tabulation
tab_result = fibonacci_tab(n)

print("\nUsing Memoization:", memo_result)
print("Using Tabulation:", tab_result)
```

Experiment 4 Output (Screenshot):

The screenshot shows the Visual Studio Code interface. The editor window displays a file named 'APP 4.py' with the following Python code:

```
26
27     memo = [-1] * (n + 1)
28     memo_result = fibonacci_memo(n, memo)
29
30     tab_result = fibonacci_tab(n)
31
32     print("\nUsing Memoization:", memo_result)
33     print("Using Tabulation:", tab_result)
```

The terminal window at the bottom shows the command prompt with the following output:

```
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> & C:\Users\
Enter n: 7

Using Memoization: 13
Using Tabulation: 13
PS C:\Users\HP\AppData\Local\Programs\Microsoft VS Code> |
```

Conclusion:

Thus, the program calculates the nth Fibonacci number efficiently by using an iterative approach with dynamic programming, significantly reducing both time and space complexity compared to naive recursive methods.